WORKOUT TRACKING SYSTEM

Therin Emmons

March 16, 2026

Dr. Robert Bailey

CSCE 548: Building Secure Software

Preface

This application allows users to track workouts, muscle groups, nutrition, recovery sessions, and equipment through a comprehensive database system. The code is written in `C++17` and `SQL`; the database can be managed using `MySQL Workbench`, and the system requires the following packages and dependencies:

```
Required Packages:
- MySQL Server (version 5.7 or higher)
- MySQL Client Library (libmysqlclient-dev)
- mysql-workbench (optional, for visual database management)
- cpp-httplib (for REST API functionality)
- libcurl (for HTTP client in CGI)
- Apache2 with CGI module (for web interface)
- pthreads (built-in on Linux, separate install on Windows)
- g++ compiler with C++17 support
```

The system follows a four-layer architecture pattern:

1. **Data Layer** – Direct database access and CRUD operations
2. **Business Layer** – Business logic and data validation
3. **Service Layer** – API interface and service orchestration
4. **Front-End Layer** – User interface via CGI web application

Setup

Setting up can be of varying degrees, but to start, go to https://github.com/temmons02/548workouttracker and download the .zip file, or use `git clone https://github.com/temmons02/548workouttracker` to get to the workout, from there, go to the `'Workout Tracker'/` directory.

0.1 Database Setup

Before running the application, initialize the MySQL database:

```
# Create database and user
mysql -u root -p
CREATE DATABASE workout_tracker;
CREATE USER 'workout_user'@'localhost'
    IDENTIFIED BY 'workout_pass';
GRANT ALL PRIVILEGES ON workout_tracker.*
    TO 'workout_user'@'localhost';
FLUSH PRIVILEGES;

# Initialize schema
mysql -u workout_user -pworkout_pass workout_tracker
    < create_tables.sql

# Load test data (optional)
mysql -u workout_user -pworkout_pass workout_tracker
    < insert_test_data.sql
```

0.2 `DataLayer/`

The Data Layer provides direct database access through the Data Access Object (DAO) pattern.

Files

- `WorkoutDAO.h/cpp` – Database access object implementing CRUD operations
- Model classes (root directory):
  - `Workout.h/cpp` – Workout entity (date, time, duration, type, calories, RPE)
  - `MuscleGroup.h/cpp` – Muscle group entity (name, description, sets, reps, weight)
  - `Nutrition.h/cpp` – Nutrition entry (food family, macros, date)
  - `Recovery.h/cpp` – Recovery session (date, duration, type, helpers)
  - `Equipment.h/cpp` – Equipment entity (name, category, target area)

Responsibilities

- Establish and manage MySQL database connections
- Execute SQL queries for INSERT, UPDATE, DELETE, SELECT operations
- Map database records to C++ object instances
- Handle database-level errors and connection failures
- Provide atomic CRUD operations for all five tables

Key Methods

```
class WorkoutDAO {
public:
    // Workout operations
    bool saveWorkout(Workout& workout);
    Workout* getWorkout(int id);
    std::vector<Workout*> getAllWorkouts();
    bool deleteWorkout(int id);

    // Similar methods for MuscleGroup, Nutrition,
    // Recovery, Equipment...
};
```

0.3 BusinessLayer/

The Business Layer contains business logic, validation rules, and orchestrates data operations.

Files

- `WorkoutManager.h/cpp` – Manages all business logic and coordinates DAO operations
- `test_crud_app.cpp` – Unit tests for CRUD operations

Responsibilities

- Implement smart save logic (auto-detects INSERT vs UPDATE based on ID)
- Validate business rules (e.g., RPE must be 1-10, duration > 0)
- Coordinate multiple DAO operations for complex transactions
- Provide helper methods for data cleanup and memory management
- Abstract data layer complexity from service layer

Key Features

```
class WorkoutManager {
public:
    // Smart save: INSERT if id=0, UPDATE otherwise
    bool saveWorkout(Workout& workout);

    // Validation built-in
    bool saveMuscleGroup(MuscleGroup& mg);

    // Memory management helper
    template<typename T>
    void cleanupVector(std::vector<T*>& vec);
};
```

0.4 `ServiceLayer/`

The Service Layer provides a clean API interface with input validation and standardized responses.

Files

- `WorkoutService.h/cpp` – Service API with validation and response formatting
- `RestApiServer.cpp` – REST API server (port 8080) using cpp-httplib
- `JsonHelper.h` – JSON serialization utilities
- `CrudFrontEnd.cpp` – Console-based REST API client

Responsibilities

- Validate all input parameters (date format, numeric ranges, required fields)
- Return standardized `ServiceResponse` objects
- Provide consistent error messages
- Expose REST API endpoints for external access
- Serialize/deserialize JSON for API communication

REST API Endpoints

```
GET  /api/workouts           - List all workouts
GET  /api/workouts/:id       - Get workout by ID
POST /api/workouts           - Create/update workout
DELETE /api/workouts/:id     - Delete workout

Similar endpoints for:
- /api/musclegroups
- /api/nutrition
- /api/recovery
- /api/equipment

Health check:
GET /health                  - Server status
```

ServiceResponse Structure

```cpp
struct ServiceResponse {
    bool success;         // Operation succeeded?
    std::string message;  // Result message
    int id;              // Created/updated record ID
};
```

0.5 `Front-EndLayer/`

The Front-End Layer provides a web-based user interface through CGI (Common Gateway Interface).

Files

- `workout_cgi.cpp` – Main CGI application handling all HTTP requests

Responsibilities

- Parse HTTP GET and POST requests
- Generate HTML output with modern styling
- Provide forms for data entry (with client-side validation)
- Display data in tables with action buttons (View, Edit, Delete)
- Handle all CRUD operations for all five tables
- Manage navigation between different sections

Supported Operations (All Tables)

- **LIST** – Display all records in a table
- **VIEW** – Show detailed information for a single record
- **INSERT** – Create new record via HTML form
- **UPDATE** – Edit existing record
- **DELETE** – Remove record with confirmation
- **FILTER** – Search/filter records by criteria (client-side)

URL Structure

```
Home page:
http://localhost/cgi-bin/workout.cgi

List records:
http://localhost/cgi-bin/workout.cgi?action=list&table=workout

Add new record:
http://localhost/cgi-bin/workout.cgi?action=insert_form&table=workout

Edit record:
http://localhost/cgi-bin/workout.cgi?action=update_form&table=workout&id=5

Delete record:
http://localhost/cgi-bin/workout.cgi?action=delete&table=workout&id=5
```

Execution

0.6 Build Process

The project uses a Makefile for compilation and deployment.

Compile All Components

```
# Build core applications
make all

# Build CGI application
make cgi

# Build everything
make full
```

Deploy CGI to Apache

```
# Install CGI to Apache
make install-cgi

# Manual deployment
sudo cp workout.cgi /usr/lib/cgi-bin/
sudo chmod +x /usr/lib/cgi-bin/workout.cgi
sudo chown www-data:www-data /usr/lib/cgi-bin/workout.cgi
```

0.7 Running Applications

Console Applications

```
# Run main demo application
make run-main

# Run CRUD tests
make run-test

# Run console app with Service Layer
make run-console
```

REST API Server

```
# Start REST API server (port 8080)
make run-server

# Or run directly
./build/rest_api_server

# Test endpoints
```

```
curl http://localhost:8080/health
curl http://localhost:8080/api/workouts
```

Web Application (CGI)

```
# Ensure Apache is running
sudo systemctl start apache2

# Access web interface
firefox http://localhost/cgi-bin/workout.cgi

# Or with curl
curl http://localhost/cgi-bin/workout.cgi
```

0.8 Testing

```
# Run all tests
make test

# View database contents
make db-view

# Clean and rebuild
make rebuild
```

Conclusion

Most of this project was 'vibe coded' using `Claude.ai`. In terms of efficiency, the code was built relatively well, but as the program got more complicated, or there were things, that were uncertain, I had to modify and patch sections of code, such that it would be compatible with my current file structure I have for my code. The other primary issue seemed to also be memory issues, which is generally common with genAI and complicated projects involving `C++`, as well as not planning ahead with projects and needing to change or add a few lines. One example includes when I was building the `BusinessLayer/`, Claude wrote a crud test app to see if the CRUD operations were working, and there was a segmentation error that was calling a `NULL` workout pointer. For Claude not thinking ahead,there was an issue with the `connect()` function, which it would try and establish another connection, despite there being one. Simple fix was to check if the connection was already present at the handle, and if not, then connect to the handle.

Appendix: File Structure

```
workout-tracker/
|__ Makefile                    # Build system
|__ create_tables.sql          # Database schema
|__ insert_test_data.sql       # Sample data
|__ Workout.h/cpp
|__ MuscleGroup.h/cpp
|__ Nutrition.h/cpp
|__ Recovery.h/cpp
|__ Equipment.h/cpp
|__ WorkoutDAO.h/cpp
|__ main.cpp
|
|__ BusinessLayer/
|   |__ WorkoutManager.h/cpp
|   |__ test_crud_app.cpp
|   |__ view_all_data.sql
|
|__ ServiceLayer/
|  |__ WorkoutService.h/cpp
|  |__ RestApiServer.cpp
|  |__ JsonHelper.h
|  |__ CrudFrontEnd.cpp
|
|__ Front-EndLayer/
|   |__ workout_cgi.cpp        # CGI web application
|
|__ build/
|    |__ workout_tracker        # Main executable
|    |__ test_crud_app         # Test suite
|    |__ rest_api_server       # REST API
|    |__ console_app           # Service Layer console
```

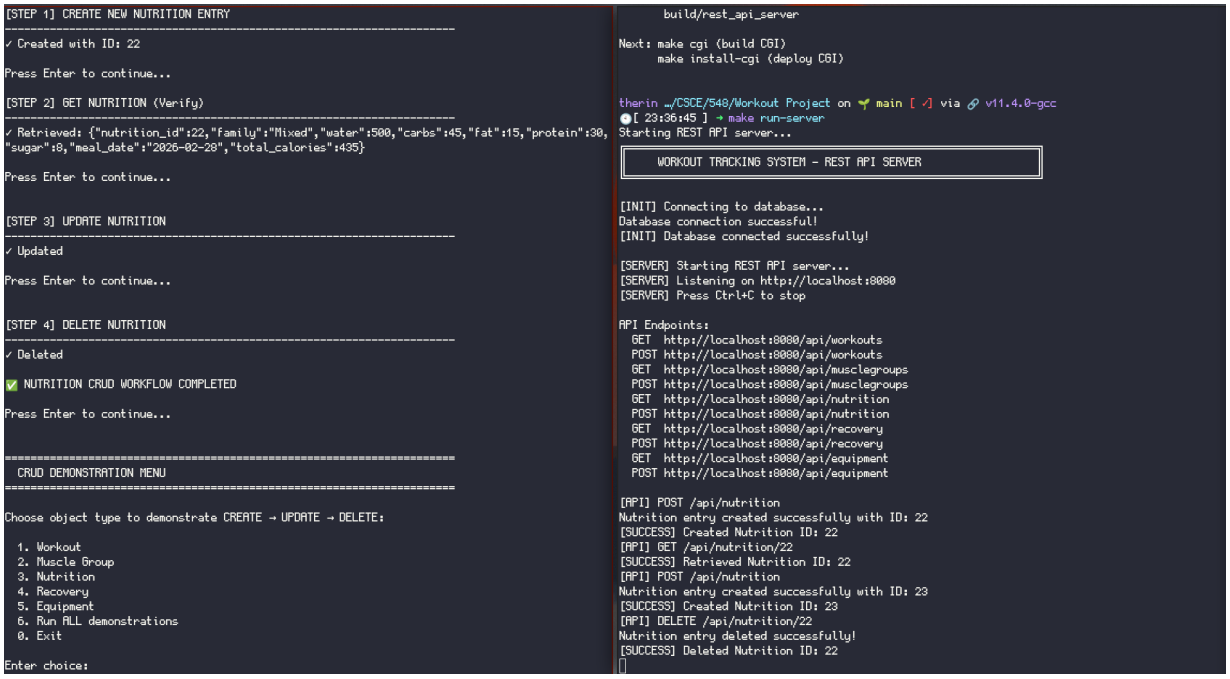

Figure 1: Testing for API server and Client for CRUD Operations

Screenshots

```
NUTRITION CRUD OPERATIONS DEMO

==========


[CREATE] Creating nutrition entry...
Nutrition entry created successfully with ID: 24

[READ BY DATE] Nutrition for 2026-01-20:
=== Nutrition Information ===
ID: 1
Family: Mixed
Date: 2026-01-20
Water: 500 ml
Carbs: 45 g
Fat: 15 g
Protein: 30 g
Sugar: 8 g
Total Calories: 435
Carb Ratio: 41.3793%
Protein Ratio: 27.5862%
Fat Ratio: 31.0345%

=== Nutrition Information ===
ID: 2
Family: Fruit
Date: 2026-01-20
Water: 200 ml
Carbs: 25 g
Fat: 0.5 g
Protein: 2 g
Sugar: 20 g
Total Calories: 112.5
Carb Ratio: 88.8889%
Protein Ratio: 7.11111%
Fat Ratio: 4%


[READ BY FAMILY] All 'Fruit' entries:
Found 2 fruit entries.

==========
```

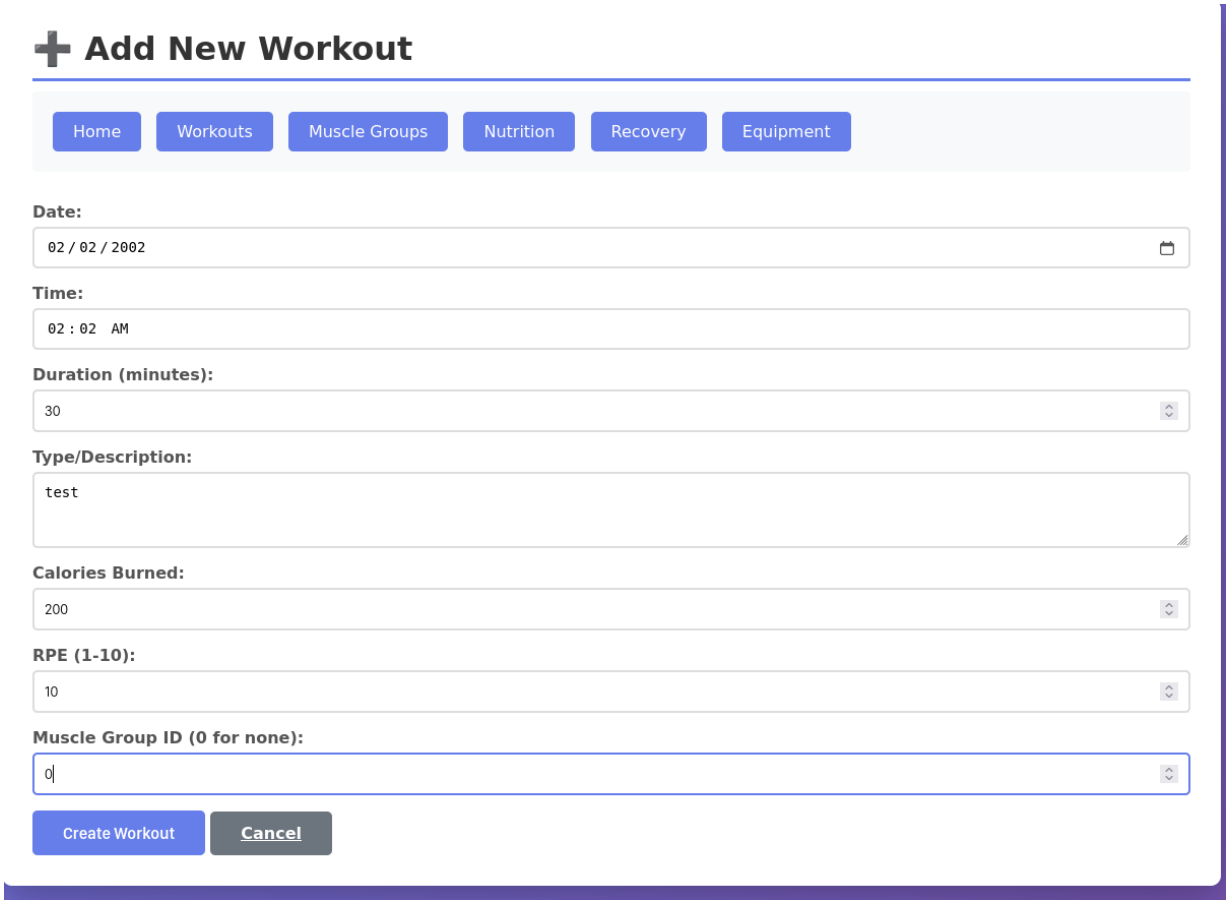

Figure 3: Testing for API server and Client for CRUD Operations

Figure 4: Testing for API server and Client for CRUD Operations

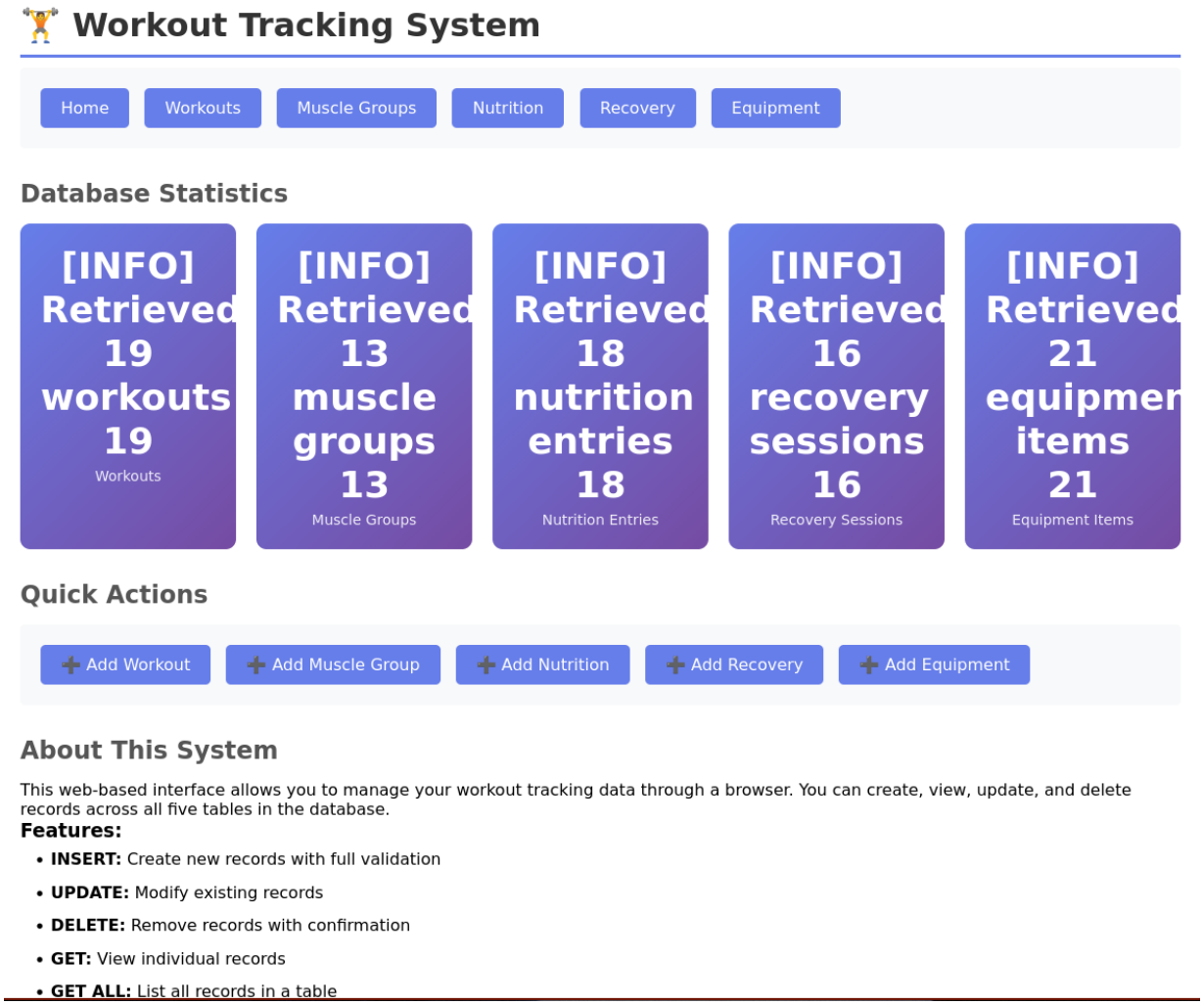

Figure 5: Testing for API server and Client for CRUD Operations

All Workouts (20)

ID	Date	Time	Duration	Type	Calories	RPE	Actions
22	2026-03-03	22:19:00	10 min	desc	500	9/10	Edit Delete
21	2026-02-28	19:00:00	45 min	API Test Workout	400	7/10	Edit Delete
16	2026-02-28	19:00:00	45 min	API Test Workout	400	7/10	Edit Delete
14	2026-02-28	19:00:00	45 min	API Test Workout	400	7/10	Edit Delete
13	2026-02-28	19:00:00	45 min	API Test Workout	400	7/10	Edit Delete
12	2026-02-28	19:00:00	45 min	API Test Workout	400	7/10	Edit Delete
11	2026-02-28	19:00:00	45 min	API Test Workout	500	9/10	Edit Delete
23	2026-01-28	08:00:00	45 min	Morning Chest Workout	400	8/10	Edit Delete
17	2026-01-28	08:00:00	45 min	Morning Chest Workout	400	8/10	Edit Delete
8	2026-01-27	17:00:00	30 min	Calf Raises and Jumps	500	9/10	Edit Delete
7	2026-01-26	06:00:00	55 min	Hip Thrusts and Glute Bridges	420	8/10	Edit Delete
6	2026-01-25	07:30:00	35 min	Planks and Russian Twists	280	7/10	Edit Delete
5	2026-01-24	06:30:00	50 min	Bicep Curls and Tricep Extensions	320	6/10	Edit Delete
4	2026-01-23	18:00:00	40 min	Shoulder Press and Lateral Raises	380	7/10	Edit Delete
10	2026-01-23	06:30:00	60 min	Full Body Circuit Training	580	8/10	Edit Delete
3	2026-01-22	06:00:00	75 min	Squats and Leg Press	600	9/10	Edit Delete
2	2026-01-21	07:00:00	45 min	Deadlifts and Pull-ups	520	9/10	Edit Delete
9	2026-01-20	18:00:00	45 min	HIIT Cardio Session	550	9/10	Edit Delete
1	2026-01-20	06:30:00	60 min	Heavy Bench Press and Cable Flyes	450	8/10	Edit Delete
25	2002-02-02	02:02:00	30 min	test2	200	10/10	Edit Delete

Figure 6: Testing for API server and Client for CRUD Operations

Figure 7: Testing for API server and Client for CRUD Operations

➕ Add New Workout [INFO] Retrieved 19 workouts

All Workouts (19)

ID	Date	Time	Duration	Type	Calories	RPE	Actions
22	2026-03-03	22:19:00	10 min	desc	500	9/10	Edit Delete
21	2026-02-28	19:00:00	45 min	API Test Workout	400	7/10	Edit Delete
11	2026-02-28	19:00:00	45 min	API Test Workout	500	9/10	Edit Delete
12	2026-02-28	19:00:00	45 min	API Test Workout	400	7/10	Edit Delete
13	2026-02-28	19:00:00	45 min	API Test Workout	400	7/10	Edit Delete
14	2026-02-28	19:00:00	45 min	API Test Workout	400	7/10	Edit Delete
16	2026-02-28	19:00:00	45 min	API Test Workout	400	7/10	Edit Delete
23	2026-01-28	08:00:00	45 min	Morning Chest Workout	400	8/10	Edit Delete
17	2026-01-28	08:00:00	45 min	Morning Chest Workout	400	8/10	Edit Delete
8	2026-01-27	17:00:00	30 min	Calf Raises and Jumps	500	9/10	Edit Delete
7	2026-01-26	06:00:00	55 min	Hip Thrusts and Glute Bridges	420	8/10	Edit Delete
6	2026-01-25	07:30:00	35 min	Planks and Russian Twists	280	7/10	Edit Delete
5	2026-01-24	06:30:00	50 min	Bicep Curls and Tricep Extensions	320	6/10	Edit Delete
4	2026-01-23	18:00:00	40 min	Shoulder Press and Lateral Raises	380	7/10	Edit Delete
10	2026-01-23	06:30:00	60 min	Full Body Circuit Training	580	8/10	Edit Delete
3	2026-01-22	06:00:00	75 min	Squats and Leg Press	600	9/10	Edit Delete
2	2026-01-21	07:00:00	45 min	Deadlifts and Pull-ups	520	9/10	Edit Delete
9	2026-01-20	18:00:00	45 min	HIIT Cardio Session	550	9/10	Edit Delete
1	2026-01-20	06:30:00	60 min	Heavy Bench Press and Cable Flyes	450	8/10	Edit Delete

Figure 8: Testing for API server and Client for CRUD Operations